

리눅스 2 - 현대 리눅스의 메모리 관리

남궁명수

현대 리눅스는 평탄한 가상 주소 공간 위에서 다단계 페이징을 중심으로 동작하고,

- VMA
- 요구 페이징
- Copy-on-Write
- 페이지 캐시
- Buddy/SLUB 할당자
- 메모리 회수
- Swap
- Huge Page
- NUMA
- cgroup

등을 결합해 메모리를 관리한다.

전체 구조를 먼저 보면 다음과 같다.

프로세스의 가상 주소 공간

↓

VMA가 주소 범위와 속성 관리

↓

다단계 페이지 테이블

↓

가상 페이지를 물리 페이지 프레임에 연결

↓

실제 RAM

RAM 부족

↓

페이지 캐시 제거 / Dirty Page 기록 / Swap

↓

그래도 부족하면 OOM Killer

[1] [세그멘테이션은 현재 어떻게 사용되는가?]

[1-1] 전통적인 세그멘테이션

운영체제 책에서 배우는 세그멘테이션은 프로그램을 의미 있는 가변 크기 영역으로 나누는 방식이다.

Code Segment
Data Segment
Heap Segment
Stack Segment

각 세그먼트는 다음 정보를 가진다.

Base: 세그먼트가 시작하는 물리 주소
Limit: 세그먼트의 크기

CPU는 논리 주소를 다음처럼 물리 주소로 변환한다.

논리 주소 = Segment 번호 + Offset
물리 주소 = Segment Base + Offset

하지만 가변 크기 세그먼트를 실제 물리 메모리에 배치하면 외부 단편화 문제가 발생한다.

[1-2] 현대 64비트 리눅스의 세그멘테이션

현대 x86-64 리눅스는 일반적인 메모리 변화에 하드웨어 세그멘테이션을 거의 사용하지 않는다.

대부분의 세그먼트를 전체 주소 공간에 걸쳐 설정하는 Flat Memory Model을 사용한다.

플랫 메모리 모델(Flat Memory Model)

컴퓨터 메모리를 깊임 없이 하나의 연속된 커다란 배열처럼 다루는 주소 지정 방식이며, 코드와 데이터를 구별하지 않고 같은 공간에 두고, 복잡한 나누기(세그먼트) 없이 쭉 이어서 쓰는 구조입니다.

Code Segment Base = 0
Data Segment Base = 0

가상 주소 $\approx$ Offset

즉, 다음과 같은 하드웨어 세그먼트로 메모리를 나누지 않는다.

Code 물리 영역
Data 물리 영역
Heap 물리 영역
Stack 물리 영역

x86-64에서 FS, GS같은 세그먼트 레지스터는 예외적으로 스레드 로컬 저장소인 LTS나 커널의 CPU별 데이터에 활용될 수 있다.

하지만 일반적인 메모리 주소 변환은 페이지 테이블이 담당한다.
ARM64처럼 x86식 세그멘테이션 자체가 없는 아키텍처도 있다.

따라서 현대 리눅스는 다음처럼 이해하면 된다.

하드웨어 세그멘테이션

→ 일반 메모리 변환에서는 거의 사용하지 않음

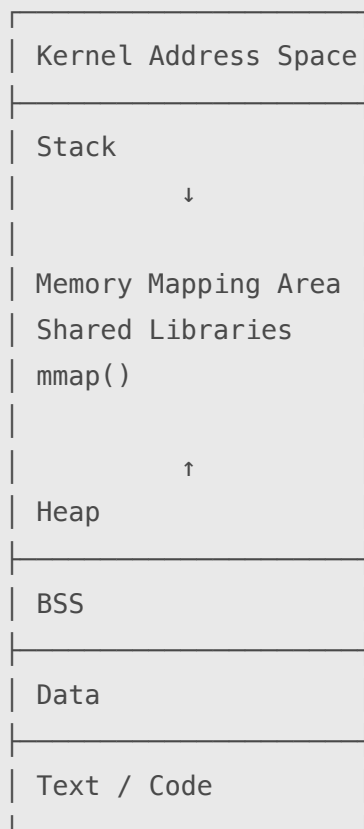
페이징

→ 실제 주소 변환과 보호의 중심

[2] [코드·데이터·힙·스택은 없어진 것인가?]

없어진 것은 아니다. 프로세스의 가상 주소 공간에는 여전히 다음과 같은 논리적인 영역이 존재한다.

높은 주소



낮은 주소

하지만 이것들은 전통적인 하드웨어 세그멘테이션의 세그먼트가 아니다.
현대 리눅스는 이러한 영역을 VMA(Virtual Memory Area)로 관리한다.

[VMA (Virtual Memory Area)]

연속된 가상 주소 범위 중 접근 권한과 용도가 동일한 영역이다.

주체:

- 리눅스 커널

직접 대상:

- 프로세스의 가상 주소 공간
- 동일한 권한과 속성을 가진 연속된 가상 주소 범위

저장 위치:

- VMA 관리 구조체인 `vm_area_struct`는 RAM의 커널 메모리에 저장
- 프로세스의 VMA들은 `mm_struct`에 연결된 Maple Tree로 관리

CPU와의 관계:

- CPU는 일반적인 주소 변환 과정에서 VMA를 직접 조회하지 않음
- CPU의 MMU는 TLB와 페이지 테이블을 이용해 주소를 변환
- 페이지 폴트가 발생하면 커널이 VMA를 조회하여 유효한 접근인지 판단

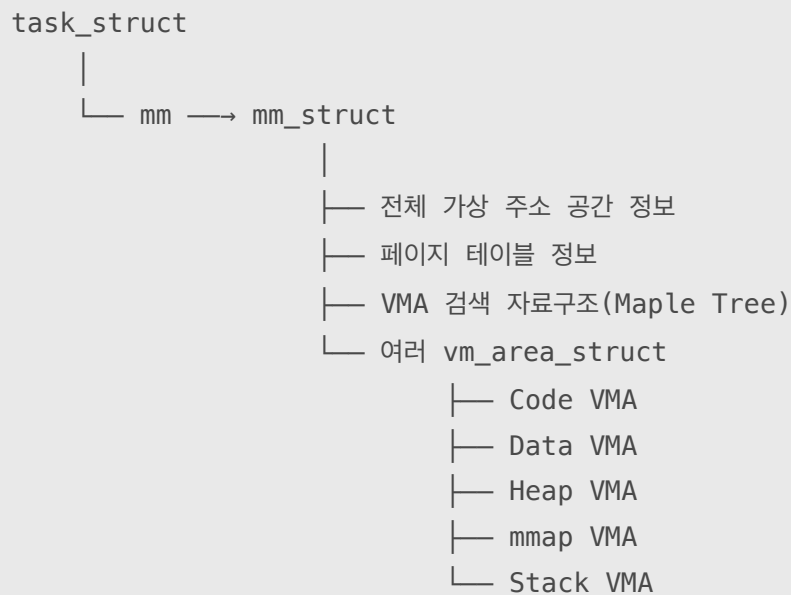
목적:

- 프로세스의 가상 주소 공간을 용도와 속성이 같은 범위로 관리
- 각 범위의 읽기·쓰기·실행 권한을 관리
- 코드, 데이터, 힙, 스택, mmap 영역 등을 구분
- 페이지 폴트가 정상적인 지연 할당인지 잘못된 메모리 접근인지 판단

예를 들어, 다음과 같다.

VMA	권한	연결 대상
실행 파일 Code	r-x	실행 파일
읽기 전용 데이터	r--	실행 파일
Data	rw-	실행 파일
Heap	rw-	익명 메모리
공유 라이브러리 Code	r-x	.so 파일
Stack	rw-	익명 메모리

각 프로세스는 mm_struct라는 구조체로 전체 가상 주소 공간을 표현하고, 각 주소 범위는 vm_area_struct로 표현한다.



즉,

mm_struct = VMA들을 관리하는 정보
 + 페이지 테이블 정보
 + 주소 공간 전체에 대한 통계와 기준 주소
 + 동기화 정보

[mm_struct의 역할]

| mm_struct는 하나의 프로세스가 사용하는 전체 가상 주소 공간을 표현한다.

단순화하면 다음과 같은 정보를 가질 수 있다.

```

struct mm_struct {
    struct maple_tree mm_mt;    // VMA들을 저장하고 검색하는 Maple Tree

    pgd_t *pgd;                // 최상위 페이지 테이블 주소

    unsigned long start_code;   // 코드 영역 시작 주소
    unsigned long end_code;     // 코드 영역 끝 주소

    unsigned long start_data;   // 데이터 영역 시작 주소
    unsigned long end_data;     // 데이터 영역 끝 주소

    unsigned long start_brk;    // 힙 시작 주소
    unsigned long brk;          // 현재 힙 끝 주소
    
```

```

unsigned long start_stack; // 스택 시작 기준 주소

unsigned long total_vm;    // 전체 가상 메모리 페이지 수

// VMA 동시 접근 제어, 메모리 통계 등의 정보
};

```

핵심은 다음 두 가지이다.

- 전체 주소 공간의 기준 정보
- 각각의 주소 범위를 나타내는 VMA 관리

[vm_area_struct의 역할]

vm_area_struct는 전체 가상 주소 공간 중에서 연속된 하나의 주소 범위와 속성을 표현한다.

```

struct vm_area_struct {
    unsigned long vm_start;    // VMA 시작 가상 주소
    unsigned long vm_end;      // VMA 끝 가상 주소

    unsigned long vm_flags;    // 읽기·쓰기·실행 등의 속성
    pgprot_t vm_page_prot;     // 페이지 보호 속성

    struct file *vm_file;      // 파일 매핑이면 연결된 파일
    struct mm_struct *vm_mm;    // 자신이 속한 mm_struct
};

```

VMA를 나누는 기준으로 단순히 코드, 힙, 스택이라는 이름이 아니라 다음 속성이다.

- 주소가 연결되어 있는가
- 접근 권한이 같은가
- 같은 파일이나 익명 메모리에 매핑되어 있는가
- 공유, 비공유 같은 매핑 속성이 같은가

속성이 달라지는 지점에서 별도의 VMA로 나뉜다.

예를 들어, 같은 실행 파일 코드 영역도 권한이나 매핑 조건이 다르면 여러 VMA가 될 수 있다.

```

00400000-00401000  r--p  프로그램 헤더
00401000-00405000  r-xp  실행 코드
00405000-00407000  r--p  읽기 전용 데이터
00407000-00408000  rw-p  전역 데이터

```

따라서 반드시 다음 처럼 1:1 대응하는 것은 아니다.

```

코드 영역 = VMA 하나
데이터 영역 = VMA 하나

```

더 정확하게 말하자면

코드·데이터·힙·스택·mmap 영역

→ 권한과 매핑 속성에 따라 하나 이상의 VMA로 표현

최신 리눅스에서는 한 프로세스의 VMA들을 과거의 Red-Black Tree 대신 Maple Tree 자료구조로 관리한다. 주소 범위 검색과 동시 접근 확장성을 개선하기 위한 변화다.

따라서 다음 둘을 구분해야 한다.

전통적인 세그멘테이션

→ CPU가 Segment Base와 Limit으로 주소 변환

현대 리눅스의 VMA

→ 커널이 가상 주소 범위의 용도와 권한을 관리

→ 실제 주소 변환은 페이지 테이블이 담당

[3] [현대 리눅스의 중심은 다단계 페이징이다]

거대한 가상 주소 공간을 효율적으로 관리하기 위해 페이지 테이블을 여러 계층으로 쪼개어 사용하는 메모리 관리 기법이다.
사용하지 않는 메모리 영역의 페이지 테이블 생성을 생략해 메모리 낭비를 줄인다.

주체:

- 리눅스 커널이 페이지 테이블을 생성하고 관리
- CPU의 MMU가 페이지 테이블을 탐색하여 주소를 변환

직접 대상:

- 프로세스의 가상 페이지와 물리 프레임 사이의 매핑 정보
- 프로세스별 페이지 테이블

저장 위치:

- 각 단계의 페이지 테이블은 RAM의 커널이 관리하는 물리 페이지에 저장
- 최근 주소 변환 결과는 CPU 내부 TLB에 캐시

CPU와의 관계:

- CPU가 가상 주소를 생성하면 MMU가 먼저 TLB를 조회
- TLB miss가 발생하면 MMU가 RAM의 다단계 페이지 테이블을 탐색
- 유효한 매핑을 찾으면 물리 주소로 변환하고 결과를 TLB에 캐시
- 유효한 매핑이 없으면 페이지 폴트를 발생시켜 커널을 실행

목적:

- 거대한 페이지 테이블을 한 번에 모두 만들지 않기 위해 사용
- 실제로 사용하는 가상 주소 범위에 필요한 하위 페이지 테이블만 생성
- 페이지 테이블이 차지하는 RAM 사용량을 절약
- 넓은 가상 주소 공간을 효율적으로 관리
- 프로세스마다 독립적인 가상 주소 공간을 제공

현대 리눅스는 프로세스가 사용하는 가상 주소를 물리 주소로 변환할 때 페이징을 사용한다.

가상 주소

↓

페이지 테이블

↓

물리 페이지 프레임

↓

RAM

가상 메모리는 고정 크기의 가상 페이지로 나뉘고, 물리 메모리 같은 크기의 페이지 프레임으로 나뉜다. 일반적인 x86-64 시스템의 기본 페이지 크기는 4KiB다.

[가상 메모리]

Virtual Page 0

Virtual Page 1

Virtual Page 2

Virtual Page 3

[물리 메모리]

Page Frame 0

Page Frame 1

Page Frame 2

Page Frame 3

가상 페이지는 물리 메모리에서 연속되어 있을 필요가 없다.

가상 페이지 0 → 물리 프레임 10

가상 페이지 1 → 물리 프레임 42

가상 페이지 2 → 물리 프레임 7

가상 페이지 3 → 물리 프레임 81

프로세스 입장에서는 가상 주소가 연속되어 있지만 실제 물리 페이지는 흩어져 있을 수 있다. 이 방식으로 외부 단편화 문제를 크게 줄이고 프로세스마다 독립된 주소 공간을 제공한다.

[4] [다단계 페이지 테이블]

64비트 가상 주소 전체를 하나의 페이지 테이블로 관리하면 페이지 테이블 자체가 지나치게 커진다. 그래서 현대 리눅스는 페이지 테이블을 계층적으로 구성한다.

리눅스는 아키텍처 독립적으로 최대 5단계 페이지 테이블 구조를 정의한다.

```
PGD(Page Global Directory 최상위)
↓
P4D(Page Level4 Directory)
↓
PUD(Page Upper Directory)
↓
PMD(Page Middle Directory)
↓
PTE(Page Table Entry 최종 물리 주소 연결)
↓
물리 페이지 프레임
```

실제 하드웨어가 4단계만 지원하면 사용하지 않는 단계는 접어서 처리한다. 5단계 페이징을 지원하고 활성화한 x86-64 시스템에서는 실제 5단계를 사용할 수 있다.

페이지 테이블의 마지막 항목인 PTE에는 다음과 같은 정보가 들어간다.

- 물리 페이지 프레임 번호
- 페이지 존재 여부
- 읽기·쓰기·실행 권한
- 사용자 모드 접근 가능 여부
- Dirty 여부
- Accessed 여부
- Swap 관련 정보

[5] [TLB를 이용한 주소 변환 가속]

CPU가 메모리를 접근할 때마다 여러 단계의 페이지 테이블을 모두 읽으면 느리다. 그래서 CPU는 최근의 주소 변환 결과를 TLB(Translation Lookaside Buffer)에 저장한다.

```
CPU가 가상 주소 접근
↓
TLB 검색
├── Hit
│   ↓
│   바로 변환
└── Miss
    ↓
    페이지 테이블 탐색
```

TLB Hit가 발생하면 페이지 테이블을 다시 탐색하지 않고 빠르게 물리 주소를 얻을 수 있다.

문맥 교환으로 다른 프로세스가 실행되면 페이지 테이블의 기준도 바뀐다.

현대 CPU는 ASID 또는 PCID 같은 식별자를 이용해 서로 다른 주소 공간의 TLB 항목을 구분함으로써 불필요한 전체 TLB 초기화를 줄일 수 있다.

[6] [VMA와 페이지 테이블의 차이]

VMA와 페이지 테이블은 서로 다른 역할을 한다.

구분	VMA	페이지 테이블
관리 주체	리눅스 커널	커널이 설정하고 MMU가 사용
관리 단위	연속된 가상 주소 범위	개별 페이지
주요 정보	용도, 권한, 파일 연결	물리 프레임, 권한, 상태 비트
예시	Heap 영역 전체	Heap 내부의 각 4KiB 페이지
자료구조	Maple Tree	다단계 페이지 테이블

예를 들어, 1GB 크기의 Heap VMA가 존재하더라도 모든 페이지가 즉시 RAM에 할당되는 것은 아니다. 프로세스가 특정 페이지를 실제로 접근할 때 필요한 물리 페이지가 할당된다.

[7] [요구 페이지징]

프로세스를 실행할 때 처음부터 모든 코드를 메모리에 올리지 않고, CPU가 실제로 필요로 할 때 해당 페이지만 물리 메모리에 적재하는 가상 메모리 관리 기법이다.

주체:

- CPU가 페이지 폴트를 발생시킴
- 리눅스 커널이 페이지 폴트를 처리하고 필요한 페이지를 RAM에 적재

직접 대상:

- 프로세스가 실제로 접근한 가상 페이지
- 해당 가상 페이지에 연결할 물리 프레임
- 가상 페이지와 물리 프레임을 연결하는 페이지 테이블 엔트리(PTE)

저장 위치:

- 사용 전 데이터는 실행 파일, 매핑된 파일 또는 Swap에 존재할 수 있음
- 익명 메모리는 최초 접근 시 빈 물리 페이지로 준비
- 적재된 페이지는 RAM의 물리 프레임에 저장
- 주소 변환 정보는 RAM의 페이지 테이블에 저장
- 최근 주소 변환 결과는 CPU 내부 TLB에 캐시

CPU와의 관계:

- CPU의 MMU가 TLB와 페이지 테이블을 이용해 주소를 변환
- PTE에 유효한 물리 프레임 매핑이 없으면 CPU가 페이지 폴트를 발생
- CPU가 커널의 페이지 폴트 처리 코드로 실행 흐름을 넘김
- 커널이 페이지를 RAM에 준비하고 PTE를 갱신
- 이후 CPU가 중단되었던 명령어를 다시 실행

목적:

- 프로세스가 실제로 사용하는 페이지만 RAM에 올리기 위해 사용
- 사용하지 않는 페이지가 RAM을 차지하지 않도록 함
- 프로그램 시작 시간을 줄임
- 제한된 RAM으로 더 많은 프로세스를 실행할 수 있게 함

현대 리눅스는 요구 페이징(Demand Paging)을 사용한다.
메모리를 요청했다고 해서 모든 물리 메모리를 즉시 할당하는 것이 아니라,
실제로 접근할 때 할당하거나 불러온다.

예를 들어, 프로그램이 1GB를 요청했다고 하자.

```
char *buffer = malloc(1024 * 1024 * 1024);
```

이 순간에는 주로 다음 작업이 수행된다.

가상 주소 범위 확보
VMA 생성 또는 확장

하지만 1GB의 물리 RAM이 즉시 모두 할당된다고 보장할 수는 없다. 이후 다음 코드가 실행된다.

```
buffer[0] = 'A';
```

해당 가상 페이지가 물리 메모리에 연결되어 있지 않다면 Page Fault가 발생한다.

```
가상 주소 접근
↓
PTE에 물리 페이지가 없음
↓
Page Fault
↓
커널이 VMA 확인
↓
유효한 접근인지 확인
↓
물리 페이지 할당
↓
PTE에 물리 페이지 연결
↓
명령어 다시 실행
```

이것이 현대 리눅스의 핵심적인 메모리 할당 흐름이다.

[8] [Page Fault의 종류]

CPU가 가상 주소로 지정된 메모리에 접근하려 할 때,
그 페이지가 현재 실제 물리 메모리에 올라와 있지 않거나 매핑되지 않은 상태일 때 발생하는 하드웨어 예외이다.

주체:

- CPU의 MMU가 문제를 감지
- CPU가 Page Fault 예외를 발생
- 리눅스 커널이 원인을 판단하고 처리

직접 대상:

- CPU가 접근하려는 가상 페이지
- 해당 가상 페이지의 페이지 테이블 엔트리(PTE)
- 필요한 경우 연결할 RAM의 물리 프레임

저장 위치:

- Page Fault 자체는 저장되는 자료구조가 아니라 CPU가 발생시키는 예외
- 가상 페이지의 매핑과 권한 정보는 RAM의 페이지 테이블에 저장
- VMA 정보는 RAM의 커널 메모리에 저장
- 필요한 데이터는 파일, Swap 또는 RAM에 있을 수 있음

CPU와의 관계:

- CPU의 MMU가 TLB와 페이지 테이블을 확인
- 페이지가 없거나 접근 권한이 맞지 않으면 CPU가 Page Fault를 발생
- CPU가 실행 중이던 명령을 중단하고 커널의 Page Fault Handler를 실행
- 커널의 처리가 성공하면 CPU가 중단된 명령을 다시 실행
- 처리할 수 없는 접근이면 커널이 프로세스에 SIGSEGV를 전달

목적:

- 현재 가상 주소 접근을 정상적으로 수행할 수 없음을 커널에 알림
- 요구 페이지를 통해 필요한 물리 페이지를 뒤늦게 할당
- Swap으로 나간 페이지를 RAM으로 복구
- Copy-on-Write를 처리
- 잘못된 주소 또는 권한 위반 접근을 차단

Page Fault가 발생했다고 해서 무조건 오류는 아니다.

[Minor Page Fault]

- 정의: 요청한 페이지가 물리 메모리(RAM)에 이미 적재되어 있으나, 현재 프로세스의 페이지 테이블에만 기록되지 않은 상태이다.
- 특징
 - 디스크 I/O 작업이 발생하지 않아 속도가 매우 빠르다.
 - 다른 프로세스가 이미 해당 페이지(예: 공유 메모리, 공통 라이브러리)를 메모리에 올려둔 경우 발생

예시는 다음과 같다.

- 새 익명 페이지 할당
- 이미 페이지 캐시에 있는 파일 페이지 연결
- Copy-on-Write 처리
- 이미 RAM에 있는 공유 페이지 매핑

[Major Page Fault]

- 정의: 요청한 페이지가 물리 메모리(RAM)에 전혀 없고 보조기억장치(HDD/SDD)의 스왑 영역이나 파일 시스템에 있는 상태이다.
- 특징
 - 디스크에서 물리 메모리로 데이터를 읽어오는 느린 디스크 I/O 작업이 필수적이다.
 - 시스템 성능 저하의 주요 원인이 될 수 있다.
- 실행 파일의 코드가 아직 RAM에 없음
- 파일 페이지가 페이지 캐시에 없음
- Swap으로 내보낸 페이지를 다시 읽어야 함

[잘못된 메모리 접근]

접근한 주소가 어떠한 유효한 VMA에도 포함되지 않거나 권한을 위반하면 정상적으로 해결할 수 없다.

유효하지 않은 주소 접근

↓

Page Fault

↓

VMA 없음 또는 권한 위반

↓

SIGSEGV

↓

기본 동작: 코어 덤프 후 프로세스 종료 가능

[9] [익명 메모리와 파일 기반 메모리]

현대 리눅스는 사용자 메모리를 크게 두 종류로 구분한다.

[익명 메모리]

디스크의 특정 파일과 연결(매핑)되지 않고, 커널이 직접 프로세스에 제공하는 순수 가상 메모리 공간이다.

주체:

- 리눅스 커널이 생성하고 관리
- 프로세스가 `malloc()`, `brk()`, 익명 `mmap()` 등을 요청하여 사용

직접 대상:

- 힙, 스택 등의 가상 메모리 영역
- 연결되는 RAM의 물리 페이지
- 가상 페이지와 물리 프레임의 매핑 정보

저장 위치:

- 사용 중인 데이터는 RAM에 저장
- RAM에서 회수해야 하면 일반적으로 Swap에 저장
- 원본이 되는 일반 파일은 존재하지 않음
- VMA와 페이지 테이블은 RAM의 커널 메모리에 저장

CPU와의 관계:

- CPU의 MMU가 TLB와 페이지 테이블을 통해 접근
- 처음 접근할 때 물리 페이지가 없다면 CPU가 Page Fault 발생
- 커널이 빈 페이지를 준비하고 페이지 테이블에 연결
- Swap으로 나간 페이지에 접근하면 Page Fault 후 RAM으로 복구

목적:

- 실행 중 새롭게 만들어지는 데이터를 저장
- 힙, 스택, 동적 객체처럼 파일에 원본이 없는 데이터를 관리
- 프로그램이 필요한 만큼 메모리를 동적으로 사용할 수 있도록 제공

특정 파일과 연결되지 않은 메모리다.

- Heap
- Stack
- 익명 `mmap()`
- 프로그램의 동적 할당 메모리

익명 메모리는 원본 파일이 없기 때문에 RAM이 부족해 회수하려면 일반적으로 Swap에 저장해야 한다.

익명 페이지

RAM → Swap

[파일 기반 메모리]

디스크에 있는 실제 파일 내용이 [페이지 캐시]나 메모리 맵을 통해 물리 메모리에 적재되어 관리되는 영역을 말한다.

주체:

- 리눅스 커널이 생성하고 관리
- 프로세스가 실행 파일을 실행하거나 `mmap()`으로 파일을 매핑하여 사용

직접 대상:

- 파일과 연결된 프로세스의 가상 메모리 영역
- Page Cache에 존재하는 RAM의 물리 페이지
- 가상 페이지와 파일 데이터 사이의 매핑

저장 위치:

- 원본 데이터는 SSD/HDD 등의 파일에 저장
- 사용 중인 파일 데이터는 RAM의 Page Cache에 저장
- VMA와 페이지 테이블은 RAM의 커널 메모리에 저장
- 변경된 페이지는 매핑 방식에 따라 파일에 다시 기록될 수 있음

CPU와의 관계:

- CPU의 MMU가 TLB와 페이지 테이블을 통해 접근
- 페이지가 RAM에 없다면 CPU가 Page Fault 발생
- 커널이 파일 데이터를 Page Cache로 읽어 온 뒤 페이지 테이블에 연결
- 이후 CPU는 일반 RAM처럼 해당 메모리에 접근

목적:

- 실행 파일이나 일반 파일의 내용을 가상 주소 공간에 연결
- `read()/write()`를 반복하지 않고 메모리 접근 방식으로 파일을 사용
- Page Cache를 통해 파일 입출력 성능을 높임
- 여러 프로세스가 동일한 파일 페이지를 공유할 수 있게 함

파일 내용과 연결된 메모리다.

- 실행 파일의 Code
- 공유 라이브러리
- `mmap()`으로 매핑한 파일
- 페이지 캐시

수정되지 않은 파일 페이지는 RAM에서 제거해도 나중에 원본 파일에서 다시 읽을 수 있다.

Clean File Page

RAM에서 제거

↓

나중에 필요하면 파일에서 다시 읽음

수정된 Dirty Page라면 파일에 먼저 기록한 후 제거해야 한다.

[10] [Copy-on-Write]

자원을 효율적으로 관리하기 위해 데이터를 즉시 복사하지 않고 공유하다가, 실제로 변경(쓰기)이 일어날 때 그제서야 비로소 복사를 수행하는 최적화 기법이다.

주체:

- 리눅스 커널이 공유 페이지와 페이지 테이블을 관리
- CPU의 MMU가 쓰기 권한 위반을 감지하고 Page Fault를 발생
- 커널이 Page Fault를 처리하면서 실제 페이지를 복사

직접 대상:

- 여러 프로세스가 공유하는 RAM의 물리 페이지
- 각 프로세스의 가상 페이지
- 가상 페이지와 물리 프레임 연결하는 PTE
- PTE의 쓰기 권한

저장 위치:

- 공유 중인 데이터는 RAM의 동일한 물리 페이지에 저장
- 복사된 데이터는 RAM의 새로운 물리 페이지에 저장
- 각 프로세스의 매핑 정보는 RAM의 페이지 테이블에 저장
- 익명 페이지가 회수되면 Swap에 저장될 수 있음

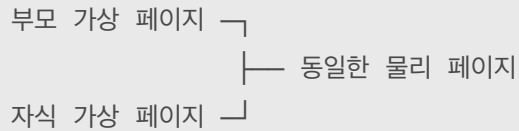
CPU와의 관계:

- CPU의 MMU가 TLB와 페이지 테이블을 통해 공유 페이지에 접근
- 읽기는 허용되므로 같은 물리 페이지를 그대로 사용
- 쓰기를 시도하면 읽기 전용 PTE 때문에 CPU가 Page Fault를 발생
- 커널이 새로운 물리 페이지를 만들고 내용을 복사
- 커널이 쓰기를 시도한 프로세스의 PTE를 새로운 페이지로 변경
- 이후 CPU가 중단됐던 쓰기 명령을 다시 실행

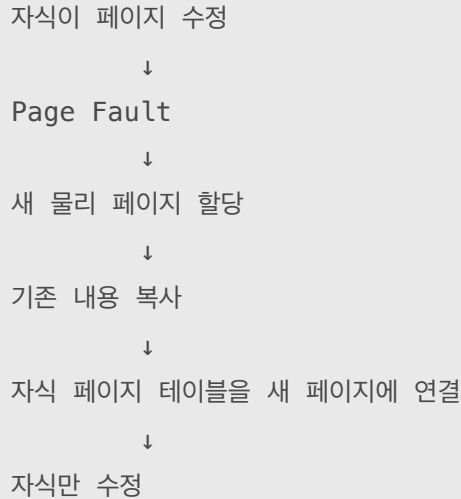
목적:

- 실제로 수정되기 전까지 물리 페이지 복사를 미룸
- fork()의 실행 시간과 RAM 사용량을 줄임
- 수정하지 않는 페이지를 부모와 자식이 계속 공유
- MAP_PRIVATE 파일 매핑에서 수정 내용을 원본 파일과 분리

fork()가 호출되면 부모 프로세스의 전체 메모리를 즉시 복사하지 않는다.
처음에는 부모와 자식이 동일한 물리 페이지를 공유한다.



페이지 테이블에는 해당 페이지가 읽기 전용으로 설정된다.
 부모나 자식이 페이지를 수정하려고 하면 쓰기 보호 Page Fault가 발생한다.



이것이 Copy-on-Write다.
 fork() 직후 execve()를 호출하는 일반적인 패턴에서 불필요한 전체 메모리 복사를 막아준다.

[11] [Page Cache]

느린 디스크 대신 빠른 주기억장치(RAM)에 파일 데이터를 임시로 저장해 입출력 속도를 높이는 커널 메모리 캐싱 메커니즘이다.

주체:

- 리눅스 커널이 생성하고 관리
- 파일 시스템과 커널의 메모리 관리 기능이 함께 사용
- 커널이 파일 데이터를 RAM에 적재하고 필요하면 디스크에 기록

직접 대상:

- SSD/HDD에 저장된 파일의 데이터
- 파일 데이터를 페이지 단위로 저장한 RAM 영역
- 최근에는 여러 페이지를 묶어 Folio 단위로 관리할 수 있음

저장 위치:

- 파일 데이터의 원본은 SSD/HDD의 파일에 저장
- 캐시된 복사본은 RAM의 Page Cache에 저장
- 어떤 파일의 어느 위치인지 나타내는 관리 정보도 RAM의 커널 메모리에 저장
- 변경된 Dirty 페이지는 RAM에 있다가 Writeback을 통해 파일에 기록

CPU와의 관계:

- Page Cache는 CPU 내부 캐시가 아니라 RAM에 존재하는 커널의 파일 캐시
- `read()`를 호출하면 커널이 Page Cache의 데이터를 사용자 메모리로 복사
- 파일을 `mmap()`한 경우 CPU가 Page Cache의 물리 페이지에 직접 매핑된 가상 주소로 접근
- `mmap()` 페이지가 아직 RAM에 없다면 CPU가 Page Fault를 발생
- 커널이 파일 데이터를 Page Cache로 읽고 페이지 테이블에 연결

목적:

- 동일한 파일 데이터를 디스크에서 반복해서 읽는 작업을 줄임
- 느린 SSD/HDD 접근 대신 RAM에서 빠르게 파일 데이터를 제공
- 여러 프로세스가 같은 파일 데이터를 공유할 수 있게 함
- 파일 쓰기를 RAM에 먼저 반영한 뒤 나중에 디스크에 기록해 성능을 높임
- `read()/write()`와 `mmap()`이 같은 파일 캐시를 공유하도록 함

현대 리눅스는 남은 RAM을 파일 데이터 캐시로 적극적으로 사용한다.

프로세스가 파일 읽기

↓

디스크에서 페이지 읽기

↓

Page Cache에 저장

↓

프로세스에 전달

같은 파일을 다시 읽으면 저장장치에 접근하지 않고 RAM의 Page Cache에서 가져올 수 있다.

두 번째 파일 읽기

↓

Page Cache Hit

↓

RAM에서 바로 읽음

따라서 `free` 명령에서 사용 가능한 RAM이 적어 보인다고 반드시 메모리가 부족한 것은 아니다. 파일 캐시로 사용된 Clean Page는 필요할 때 비교적 쉽게 회수할 수 있다.

[12] [물리 메모리는 Buddy Allocator로 관리한다]

물리 메모리를 페이지 단위로 나누어 관리하고 외부 단편화를 줄이기 위해 2의 거듭제곱 크기로 메모리를 할당 및 병합하는 리눅스 커널의 기본 물리 메모리 관리 알고리즘이다.

주체:

- 리눅스 커널의 물리 메모리 관리 기능
- 커널이 물리 페이지를 요청하거나 반환할 때 동작

직접 대상:

- RAM의 사용 가능한 물리 페이지 프레임
- 2의 거듭제곱 개수로 묶인 연속 물리 페이지
- 예: 1개, 2개, 4개, 8개 페이지

저장 위치:

- 실제 할당 대상은 RAM의 물리 페이지
- 빈 페이지 블록 목록과 관리 정보도 RAM의 커널 메모리에 저장
- 각 물리 페이지의 상태는 `struct page` 등의 자료구조로 관리

CPU와의 관계:

- CPU 내부에서 동작하는 하드웨어 기술은 아님
- CPU가 리눅스 커널의 Buddy Allocator 코드를 실행
- 할당된 물리 페이지는 페이지 테이블을 통해 커널이나 프로세스의 가상 주소에 연결
- 이후 CPU의 MMU가 해당 가상 주소를 물리 주소로 변환하여 접근

목적:

- RAM의 물리 페이지를 빠르게 할당하고 반환
- 연속된 물리 페이지가 필요할 때 제공
- 반환된 인접 페이지 블록을 병합하여 외부 단편화를 줄임
- SLUB, Page Cache, 페이지 테이블 등에 필요한 물리 페이지를 공급

물리 RAM은 페이지 프레임으로 나뉜다.

리눅스는 연속된 물리 페이지를 할당하기 위해 Buddy Allocator(페이지 할당자)를 사용한다.

[주요 동작 방식]

- 2의 제곱 단위 분할: 요청된 메모리 크기에 맞추어 가장 가까운 2의 제곱 크기 블록을 찾고, 큰 블록만 있다면 반으로 쪼개어 할당한다.
- 짝궁(Buddy) 블록 병합: 메모리 해제 시, 인접한 짝궁 블록이 둘 다 비어 있으면 하나의 큰 블록으로 다시 합쳐서 외부 단편화를 줄인다.

Buddy Allocator는 연속 페이지를 2의 거듭제곱 단위로 관리한다.

Order 0: 1 Page
Order 1: 2 Pages
Order 2: 4 Pages
Order 3: 8 Pages
Order 4: 16 Pages

4KB 페이지를 사용하는 경우 다음과 같다.

Order	페이지 수	크기
0	1	4KiB
1	2	8KiB
2	4	16KiB
3	8	32KiB
9	512	2MiB

큰 블록만 남아 있는데 작은 블록이 필요하면 절반으로 나눈다.

8페이지 블록

↓ 분할

4페이지 + 4페이지

↓ 분할

2페이지 + 2페이지

해제된 두 블록이 서로 Buddy 관계라면 다시 합친다.

2페이지 + 2페이지

↓ 병합

4페이지

최신 커널에서는 자주 사용하는 작은 페이지 할당을 먼저 CPU별 Per-CPU Pageset에서 처리하고, 부족하면 전역적인 Buddy Allocator를 사용한다. 멀티코어 환경에서 락 경쟁을 줄이기 위한 구조다.

[13] [물리 메모리 Zone과 NUMA Node]

[메모리 Zone]

하드웨어 장치의 제약 사항이나 사용 목적에 따라 물리 메모리(RAM) 전체 영역을 작은 단위로 나눠 관리하는 구역이다.

커널은 하드웨어 제약 극복과 메모리 단편화를 최소화하기 위해 존 단위를 사용한다.

주체:

- 리눅스 커널의 물리 메모리 관리자
- Buddy Allocator가 Zone별로 물리 페이지를 관리하고 할당

직접 대상:

- RAM의 물리 페이지 프레임
- 물리 주소 범위와 사용 목적에 따라 분류된 페이지 그룹

저장 위치:

- Zone이 관리하는 실제 페이지는 RAM에 존재

- 각 Zone의 빈 페이지 목록과 상태 정보도 RAM의 커널 메모리에 저장
- 커널은 struct zone 자료구조로 각 Zone을 표현

CPU와의 관계:

- CPU 내부의 하드웨어 기능은 아님
- CPU가 커널의 Zone 및 Buddy Allocator 코드를 실행
- 일반 CPU 메모리 접근뿐 아니라 장치의 DMA 주소 제한을 고려하여 Zone을 선택
- 할당된 물리 페이지가 페이지 테이블에 매핑되면 CPU의 MMU가 주소를 변환하여 접근

목적:

- 하드웨어마다 접근 가능한 물리 주소 범위가 다른 문제를 처리
- DMA 장치가 접근할 수 있는 낮은 주소의 물리 페이지를 따로 관리
- 특별한 용도의 페이지가 일반 할당으로 먼저 소진되는 것을 방지
- 물리 페이지 할당과 메모리 회수를 Zone 단위로 관리

리눅스는 물리 메모리를 용도와 하드웨어 제약에 따라 Zone으로 나눈다.

대표적인 Zone은 다음과 같다.

- ZONE_DMA: DMA 제약이 있는 장치용
- ZONE_DMA32: 32비트 주소만 사용하는 DMA 장치용
- ZONE_NORMAL: 일반적인 메모리
- ZONE_HIGHMEM: 주로 일부 32비트 시스템에서 사용
- ZONE_MOVABLE: 이동 가능한 페이지 중심

64비트 일반 시스템에서는 대표적인 RAM이 ZONE_NORMAL에 포함된다.

[NUMA]

멀티 프로세서 시스템에서 CPU가 메모리에 접근할 때 거리에 따라 속도가 달라지는 구조와 이를 관리하는 리눅스 커널의 기능이다.

CPU 소켓별로 가까운 로컬 메모리와 먼 리모트 메모리가 나뉘며, 리눅스는 성능 향상을 위해 가능한 가까운 메모리를 먼저 할당한다.

주체:

- 하드웨어가 NUMA 구조를 제공
 - CPU 소켓
 - 메모리 컨트롤러
 - CPU 사이의 연결망
- 리눅스 커널이 NUMA 구조를 인식하고 프로세스와 메모리 배치를 관리

직접 대상:

- 여러 CPU와 RAM을 묶은 NUMA Node
- 프로세스와 스레드가 실행될 CPU
- 프로세스에 할당할 RAM의 물리 페이지

저장 위치:

- 실제 프로그램 데이터와 물리 페이지는 각 NUMA Node에 연결된 RAM에 저장
- NUMA Node, Zone, 페이지 상태 등의 관리 정보는 RAM의 커널 메모리에 저장

CPU와의 관계:

- CPU가 자신과 가까운 Node의 RAM에 접근하면 Local Memory Access
- CPU가 다른 Node의 RAM에 접근하면 Remote Memory Access
- Remote 접근은 CPU 간 연결망을 지나므로 일반적으로 더 느림
- 커널은 가능한 한 스레드와 해당 스레드가 사용하는 메모리를 같은 Node에 배치

목적:

- CPU와 RAM이 많은 시스템에서 메모리 접근 병목을 줄임
- 모든 CPU가 하나의 메모리 경로를 공유하는 문제를 완화
- CPU가 가까운 RAM을 사용하도록 하여 접근 지연을 줄임
- 여러 CPU 소켓을 사용하는 대형 시스템의 확장성을 높임

Non-Uniform Memory Access, 불균일 기억 장치 접근은 멀티 프로세서 컴퓨터에서 CPU가 메모리에 접근하는 속도가 물리적 위치에 따라 달라지는 [불균일 메모리 구조] 및 [메모리 관리 기법]이다.

- 로컬 메모리: 각 CPU에 직접 연결된 가까운 메모리로, 접근 속도가 매우 빠르다.
- 원격 메모리: 다른 CPU에 연결된 먼 메모리로, 버스를 거쳐야 하므로 접근 속도가 상대적으로 느리다.
- 병목 현상 완화: 모든 CPU가 하나의 공용 버스의 메모리를 공유하는 UMA(Uniform Memory Access) 방식의 한계를 극복하고, 대규모 시스템에서 성능 저하를 줄인다.

NUMA 시스템에서는 메모리가 CPU와의 거리에 따라 여러 Node로 구분된다.

NUMA Node 0

├ CPU 0, CPU 1
└ Local Memory 0

NUMA Node 1

├ CPU 2, CPU 3
└ Local Memory 1

CPU 0이 Node 0 메모리에 접근하는 것은 빠르지만 Node 1 메모리에 접근하면 상대적으로 느릴 수 있다.

리눅스는 가능하면 프로세스가 실행되는 CPU와 가까운 NUMA Node에서 메모리를 할당한다. 이를 위해 First-Touch 정책, NUMA Balancing, 메모리 정책 등을 사용한다.

[14] [SLUB Allocator]

리눅스 커널에서 자주 사용하는 작은 크기의 객체(task_struct, inode 등을 효율적으로 할당하고 해제하기 위한 핵심 메모리 관리 메커니즘이다.
기존 SLAB 할당자의 복잡한 큐 구조를 제거하여 대규모 멀티코어 시스템에서 성능과 확장성을 크게 높인 모던 리눅스의 기본 메모리 할당자이다.

주체:

- 리눅스 커널
- 커널이 작은 커널 객체를 할당하고 반환할 때 동작
- CPU별로 준비된 객체를 활용해 빠르게 할당할 수 있음

직접 대상:

- 커널이 사용하는 작은 객체
- 예: task_struct, inode, dentry, 파일 관리 구조체
- Buddy Allocator에서 받은 물리 페이지를 작은 객체 단위로 나누어 관리

저장 위치:

- 실제 객체 데이터는 RAM에 저장
- 객체를 담는 Slab도 RAM의 하나 이상의 물리 페이지로 구성
- Slab과 빈 객체 등의 관리 정보는 RAM의 커널 메모리에 저장
- 커널 코드에는 객체의 커널 가상 주소가 반환됨

CPU와의 관계:

- CPU 하드웨어가 직접 관리하는 기술은 아님
- CPU가 리눅스 커널의 SLUB Allocator 코드를 실행
- CPU별로 할당 가능한 객체를 관리하여 CPU 사이의 Lock 경합을 줄임
- 반환된 커널 가상 주소는 MMU가 페이지 테이블을 통해 물리 주소로 변환

목적:

- 페이지보다 작은 커널 객체를 효율적으로 할당
- Buddy Allocator에서 객체마다 페이지를 할당할 때 발생하는 메모리 낭비 방지
- 같은 종류와 크기의 객체를 재사용하여 할당·해제 속도를 높임
- CPU별 객체 관리로 멀티코어 환경의 Lock 경합을 줄임
- 커널 객체의 반복적인 초기화 비용을 줄일 수 있음

Buddy Allocator는 페이지 단위로 물리 메모리를 할당한다.

하지만 커널은 다음처럼 페이지보다 작은 객체를 많이 생성한다.

- task_struct
- inode
- dentry
- 소켓 구조체
- 네트워크 버퍼
- 파일 관련 구조체

작은 커널 객체 하나를 만들 때마다 4KB 페이지 전체를 사용하면 메모리가 낭비된다.

그래서 현대 리눅스는 주로 SLUB Allocator를 사용한다.

Buddy Allocator에서 페이지 확보

↓

페이지를 동일한 크기의 객체로 나눔

↓

커널 객체에 할당

예를 들어, 4KB 페이지를 256바이트 객체로 나누면 다음과 같다.

4KiB Page

|— 256B 객체

|— 256B 객체

|— 256B 객체

|— ...

역할을 구분하면 다음과 같다.

Buddy Allocator

→ 물리 페이지 단위 할당

SLUB Allocator

→ 페이지보다 작은 커널 객체 할당

사용자 프로그램이 malloc()이 SLUB에서 직접 메모리를 받는 것은 아니다.

SLUB은 주로 커널 내부 객체를 위한 할당자다.

[15] [kmalloc과 vmalloc]

커널 내부에서 메모리를 요청하는 방식도 나뉜다.

[kmalloc()]

kmalloc()은 물리적으로 연속된 메모리를 제공한다.

가상 주소: 연속

물리 주소: 연속

작은 커널 객체나 물리적 연속성이 필요한 버퍼 등에 적합하다.

[vmalloc()]

vmalloc()은 가상 주소는 연속되어 있지만 물리 페이지는 흩어져 있을 수 있다.

연속된 커널 가상 주소

V0 → Physical Frame 10

V1 → Physical Frame 82

V2 → Physical Frame 31

큰 가상 연속 영역이 필요하지만 물리적으로 연속될 필요가 없을 때 사용한다.

[16] [Huge Page와 최신 THP]

기본 4KB 페이지를 사용하면 큰 메모리를 다루는 프로그램은 매우 많은 페이지 테이블 항목과 TLB 항목을 필요로 한다.

예를 들어, 1GB 메모리를 4KB 페이지로 관리하면 다음과 같다.

$1\text{GiB} / 4\text{KiB} = 262,144\text{개 페이지}$

2MB Huge Page를 사용하면 다음과 같다.

$1\text{GiB} / 2\text{MiB} = 512\text{개 페이지}$

x86-64에서는 대표적으로 다음 크기를 사용할 수 있다.

- 4KiB 기본 페이지
- 2MiB Huge Page
- 1GiB Huge Page

Huge Page의 장점은 다음과 같다.

- 페이지 테이블 크기 감소
- TLB Miss 감소
- 대용량 메모리 접근 성능 개선

리눅스의 Huge Page 방식은 크게 두 가지다.

[HugeTLB]

- 리눅스 커널에서 표준 크기보다 훨씬 큰 메모리 페이지를 관리하고 사용하기 위한 서브 시스템
- 관리자가 미리 고정된 크기로 할당해 두고 애플리케이션이 직접 사용하며 성능이 안정적이다.

이름의 본질은 리눅스 커널 개발자들이 이 기능을 만들 때,
“CPU 내부의 TLB 캐시 효율을 극대화하기 위해 RAM에 크게 잡은 페이지”라는 의미로 이름을 지었다.

주체:

- 리눅스 커널이 Huge Page를 예약하고 관리
- 관리자 또는 사용자 프로그램이 Huge Page 사용을 설정·요청
- CPU의 MMU가 Huge Page의 주소 변환을 수행

직접 대상:

- RAM의 연속된 대용량 물리 페이지
- 프로세스의 큰 가상 페이지
- 큰 가상 페이지와 큰 물리 페이지를 연결하는 페이지 테이블 엔트리

저장 위치:

- Huge Page의 실제 데이터는 RAM에 저장
- 예약된 Huge Page Pool도 RAM의 일부
- Huge Page 매핑 정보는 RAM의 페이지 테이블에 저장
- 최근 주소 변환 결과는 CPU 내부의 TLB에 캐시

CPU와의 관계:

- CPU의 MMU가 2MiB·1GiB 등의 큰 페이지 주소 변환을 지원
- CPU는 Huge Page 주소 변환 결과를 TLB에 캐시
- TLB 엔트리 하나가 넓은 메모리 범위를 담당
- HugeTLB 자체는 CPU 내부 기술이 아니라 리눅스 커널의 메모리 관리 기능

목적:

- TLB 엔트리 하나가 담당하는 메모리 범위를 넓힘
- TLB Miss를 줄임
- 다단계 페이지 테이블의 항목 수와 탐색 단계를 줄임
- 대용량 메모리에 반복 접근하는 프로그램의 성능을 높임
- 미리 예약한 Huge Page를 통해 비교적 예측 가능한 대용량 페이지를 제공

[Transparent Huge Pages]

- 대형 페이지를 커널이 백그라운드에서 자동으로 할당하고 관리해 주는 기능
- 커널이 일반 페이지들을 자동으로 큰 페이지로 합치거나 필요한 경우 다시 작은 페이지로 나눈다.

512개의 4KiB 페이지

↓ Collapse

1개의 2MiB Huge Page

최근 커널의 THP는 단순한 2MB PMD 크기뿐 아니라 더 다양한 크기의 Multi-size THP(mTHP)와 페이지 크기 승격, 강등을 발전시키고 있다.

다만 실제 활성화 정책과 크기는 커널 버전, 아키텍처, 배포판 설정에 따라 달라진다.

주체:

- 리눅스 커널이 자동으로 Huge Page를 생성하고 관리
- 페이지 폴트 처리 과정이나 khugepaged 커널 스레드가 큰 페이지를 구성
- CPU의 MMU가 Huge Page의 실제 주소 변환을 수행

직접 대상:

- 프로세스의 가상 메모리 영역
- RAM의 연속된 대용량 물리 페이지
- 여러 개의 일반 페이지와 해당 페이지 테이블 매핑
- 주로 익명 메모리에 적용되며, 조건에 따라 일부 파일·공유 메모리 매핑에도 적용 가능

저장 위치:

- 실제 데이터는 RAM의 Huge Page에 저장
- Huge Page 매핑 정보는 RAM의 페이지 테이블에 저장
- 최근 주소 변환 결과는 CPU 내부의 TLB에 캐시
- HugeTLB와 달리 전용 Huge Page Pool을 미리 예약하지 않고 일반 RAM에서 동적으로 확보

CPU와의 관계:

- CPU의 MMU가 2MiB 등의 큰 페이지 주소 변환을 지원
- Huge Page 주소 변환 결과를 CPU 내부 TLB에 캐시
- TLB 엔트리 하나가 2MiB 같은 넓은 범위를 담당
- THP 자체는 CPU 기능이 아니라 리눅스 커널의 자동 메모리 관리 기능

목적:

- 프로그램을 수정하지 않고 자동으로 Huge Page를 사용
- TLB Miss를 줄임
- 페이지 테이블 크기와 탐색 비용을 줄임
- 대용량 메모리를 연속적으로 사용하는 프로그램의 성능을 높임

둘 다 결국 큰 페이지를 사용하지만, 서로 독립적인 방식이다.

구분	THP	HugeTLB
정식 명칭	Transparent Huge Pages	HugeTLB Pages
관리 주체	리눅스 커널이 자동 관리	사용자가 미리 예약하고 명시적으로 사용
페이지 확보	일반 4KiB 페이지를 묶거나 처음부터 큰 페이지로 할당	부팅 또는 실행 중 HugeTLB 전용 풀에 예약
애플리케이션 수정	일반적으로 불필요	mmap() 등의 명시적인 요청 필요
메모리 부족 시	4KiB 페이지로 폴백 가능	예약된 HugeTLB 페이지가 없으면 할당 실패
스왑	분할 후 가능	일반적으로 스왑 불가

관계는 다음과 같다.

큰 페이지 사용 기술

└─ THP

| └─ 커널이 일반 메모리를 자동으로 큰 페이지로 관리

|

└─ HugeTLB

 └─ 미리 예약한 전용 큰 페이지를 프로그램이 명시적으로 사용

[17] [Folio와 Large Folio]

Folio(struct folio)는 메모리를 관리할 때 쓰는 새로운 데이터 구조체이다.

하나 이상의 연속된 물리 페이지를 묶어서 나타내며,

기존의 메모리 관리 단위인 struct page를 보완하고 대체하기 위해 만들어졌다.

주체:

- 리눅스 커널
- 커널의 메모리 관리 및 파일 시스템 코드가 Folio 단위로 메모리를 관리

직접 대상:

- 하나 이상의 연속된 물리 페이지
- 동일한 수명과 상태를 함께 관리할 수 있는 페이지 묶음
- Page Cache, 익명 메모리 등에 사용되는 RAM 페이지

저장 위치:

- Folio가 나타내는 실제 데이터는 RAM의 물리 페이지에 저장
- Folio의 상태, 참조 횟수, Dirty 여부 등의 관리 정보도 RAM의 커널 메모리에 저장
- Folio는 기존 struct page를 기반으로 표현되는 커널 자료구조

CPU와의 관계:

- CPU 내부의 하드웨어 기능이 아님
- CPU가 리눅스 커널의 Folio 관리 코드를 실행
- CPU의 MMU는 Folio를 직접 인식하지 않고 페이지 테이블 매핑만 사용
- Folio가 큰 페이지로 매핑된 경우 TLB 효율이 좋아질 수 있지만, 모든 Folio가 Huge Page인 것은 아님

목적:

- 여러 페이지를 하나의 단위로 효율적으로 관리
- 페이지마다 반복하던 Lock, 참조 횟수 관리, 상태 확인 비용을 줄임
- Page Cache에서 여러 페이지를 한꺼번에 읽고 쓰기 쉽게 만듦
- 복합 페이지의 Head Page와 Tail Page를 잘못 다루는 문제를 줄임
- 큰 메모리 단위 관리를 위한 일관된 커널 인터페이스 제공

최근 리눅스 메모리 관리 코드에서는 단일 struct page만을 중심으로 다루는 방식에서 Folio 추상화를 적극적으로 사용하고 있다.

Folio는 다음과 같이 이해할 수 있다.

하나 이상의 연속된 페이지를 하나의 메모리 단위로 다루기 위한 커널 내부 추상화

[기존 관점]

Page + Page + Page + Page

[Folio 관점]

여러 페이지를 묶은 Folio

Folio의 목적은 다음과 같다.

- 페이지 캐시 처리 효율 개선
- 여러 페이지를 한꺼번에 관리
- struct page의 여러 의미에서 발생하는 혼란 감소
- Large Folio와 Multi-size THP 지원 기반
- 락과 메타데이터 처리 횟수 감소

이것은 프로세스에 보이는 새로운 주소 변환 방식이라기보다는 현대 리눅스 커널 내부 메모리 관리 구조의 발전이다.

[18] [메모리가 부족하면 Page Reclaim(리클레임, 회수)을 수행한다]

시스템의 물리 메모리(RAM)가 부족해질 때 사용하지 않는 메모리 페이지를 찾아내어 비우고, 이를 다른 프로세스나 커널이 재사용할 수 있도록 확보하는 리눅스 커널의 핵심 메모리 관리 기능이다.

주체:

- 리눅스 커널의 메모리 관리 기능
- 백그라운드에서는 kswapd 커널 스레드가 수행
- 메모리가 급하게 필요하면 할당을 요청한 프로세스가 Direct Reclaim을 수행할 수도 있음

직접 대상:

- RAM을 차지하고 있는 물리 페이지
- Page Cache의 파일 기반 페이지
- 프로세스가 사용하는 익명 페이지
- 최근에 적게 사용된 회수 가능한 페이지

저장 위치:

- 회수 전 데이터는 RAM에 존재
- 깨끗한 파일 페이지의 원본은 SSD/HDD의 파일에 존재

- Dirty 파일 페이지는 파일에 Writeback
- 익명 페이지는 필요하면 Swap 파일·파티션 또는 zram 등에 저장
- 페이지 사용 상태와 회수 목록은 RAM의 커널 메모리에 저장

CPU와의 관계:

- CPU 내부의 하드웨어 기술은 아님
- CPU가 리눅스 커널의 Page Reclaim 코드를 실행
- 페이지가 회수되면 관련 PTE가 변경되고 TLB 항목이 무효화될 수 있음
- 회수된 페이지에 다시 접근하면 CPU가 Page Fault를 발생
- 커널이 파일이나 Swap에서 페이지를 다시 RAM으로 가져옴

목적:

- RAM이 부족할 때 사용 가능한 물리 페이지를 확보
- 새로운 메모리 할당 요청을 처리
- 사용 빈도가 낮은 페이지를 제거하고 활발하게 사용하는 페이지를 RAM에 유지
- OOM Killer 실행 가능성을 줄임

RAM이 부족해지면 리눅스는 사용 빈도가 낮은 페이지를 찾아 회수한다.

기본적인 우선순위는 다음과 같이 이해할 수 있다.

1. 다시 읽을 수 있는 Clean File Page 제거
2. Dirty File Page를 저장장치에 기록한 후 제거
3. 익명 페이지를 Swap으로 이동
4. 그래도 확보하지 못하면 OOM 처리

커널의 kswapd는 백그라운드에서 메모리를 회수한다.

할당 요청 중 즉시 메모리가 부족하면 해당 프로세스가 직접 회수 작업에 참여하는 Direct Reclaim이 발생할 수도 있다.

[19] [LRU와 최신 Multi-Gen LRU]

[LRU]

LRU(Least Recently Used)는 가장 오랫동안 사용되지 않은 페이지부터 회수하는 방식이다. 리눅스의 전통적인 페이지 회수 방식은 페이지의 최근 사용 여부를 바탕으로 주로 Active List와 Inactive List를 관리한다.

자주 사용하는 Hot Page

↓

Active List

오래 사용하지 않은 Cold Page

↓

Inactive List

↓

회수 후보

메모리가 부족해지면 커널은 주로 Inactive List에 있는 페이지를 검사하여 회수한다.

하지만 Active/Inactive라는 비교적 단순한 구분만으로는 페이지가 얼마나 최근에 사용되었는지 세밀하게 구분하기 어렵다.

메모리가 크고 페이지 수가 많은 시스템에서는 페이지를 검사하고 분류하는 비용도 커질 수 있다.

[Multi-Gen LRU(MGLRU)]

MGLRU는 기존 LRU 기반 페이지 회수 방식을 확장하여 페이지를 여러 세대(Generation)로 분류하는 기술이다. Linux 6.1에서 메인라인 커널에 병합되었다.

주체:

- 리눅스 커널의 메모리 관리 기능
- Page Reclaim 과정에서 커널이 페이지의 세대를 관리하고 회수 대상을 선택

직접 대상:

- RAM에 존재하는 회수 가능한 물리 페이지
- Page Cache의 파일 기반 페이지
- 프로세스의 익명 페이지
- 페이지의 최근 접근 기록과 세대 정보

저장 위치:

- 실제 페이지 데이터는 RAM에 저장
- 페이지의 세대와 LRU 목록 등의 관리 정보도 RAM의 커널 메모리에 저장
- 페이지 접근 여부는 페이지 테이블의 Accessed 비트 등의 정보를 활용

CPU와의 관계:

- CPU의 MMU는 페이지에 접근하면서 페이지 테이블에 접근 흔적을 남길 수 있음

- 커널이 페이지 테이블과 접근 정보를 조사하여 최근 사용된 페이지를 판단
- MGLRU 자체는 CPU 하드웨어 기술이 아니라 커널의 Page Reclaim 알고리즘
- 회수된 페이지에 다시 접근하면 CPU가 Page Fault를 발생

목적:

- 최근에 사용한 페이지와 오래전에 사용한 페이지를 더 정확하게 구분
- 사용 중인 Working Set을 RAM에 유지
- 사용 빈도가 낮은 페이지를 우선 회수
- 잘못된 페이지 회수로 인한 반복적인 Page Fault와 Thrashing 감소
- 대용량 메모리에서 Page Reclaim의 탐색 효율을 높임

오래된 세대

Generation N

↓

Generation N+1

↓

Generation N+2

↓

Generation N+3

최근 세대

- 오래된 세대: 오랫동안 접근되지 않은 페이지 -> 우선 회수 후보
- 최근 세대: 최근 접근된 페이지 -> 페이지에 유지할 가능성이 큼

커널은 페이지 테이블의 Accessed bit와 같은 정보를 검사하여 최근 접근된 페이지를 더 젊은 세대에 포함한다. 메모리를 회수할 때는 가장 오래된 세대의 페이지부터 우선적으로 검사한다.

페이지가 접근할 때마다 즉시 세대 사이를 이동하는 것이 아니라, 커널이 접근 정보를 주기적으로 확인하여 세대를 분류한다.

장점은 다음과 같다.

- 페이지가 언제 사용되었는지 더 세밀하게 구분
- 실제로 사용되지 않은 페이지를 더 정확하게 회수
- 자주 사용되는 페이지가 잘못 회수되는 현상 감소
- 불필요한 스왑과 페이지 재로딩 감소
- 메모리 압박 상황에서 쓰레싱과 응답 지연 완화
- 페이지 회수 과정의 CPU 사용량 감소 가능

실제 MGLRU 사용 여부는 커널 버전, 빌드 옵션, 배포판 설정 및 런타임 설정에 따라 달라질 수 있다.

[20] [Swap과 zswap]

[Swap]

주체:

- 리눅스 커널이 Swap-in·Swap-out을 수행
- 관리자 또는 사용자 공간 도구가 Swap 영역을 생성하고 활성화
- Page Reclaim이 회수할 익명 페이지를 선택

직접 대상:

- RAM에 있는 익명 메모리 페이지
- 힙, 스택, 익명 mmap() 등의 데이터
- Swap으로 이동한 페이지의 PTE

저장 위치:

- 일반적으로 SSD/HDD의 Swap 파티션 또는 Swap 파일
- Swap-out 전에는 RAM에 저장
- Swap으로 이동하면 PTE에 물리 프레임 대신 Swap 위치 정보가 기록됨

CPU와의 관계:

- CPU 내부의 기술은 아님
- CPU가 커널의 Swap 처리 코드를 실행
- Swap으로 이동한 페이지에 접근하면 CPU가 Page Fault를 발생
- 커널이 페이지를 RAM으로 가져오고 PTE를 갱신

목적:

- RAM이 부족할 때 익명 페이지를 다른 저장 공간으로 이동
- RAM의 물리 페이지를 확보
- 제한된 RAM으로 더 많은 메모리 사용을 가능하게 함
- 메모리 부족으로 OOM Killer가 빠르게 동작하는 것을 완화

익명 페이지는 원본 파일이 없으므로 RAM에서 그냥 제거할 수 없다.

따라서 Swap이 활성화되어 있다면 저장장치의 Swap 공간으로 내보낼 수 있다.

RAM의 익명 페이지

↓ Swap Out

SSD 또는 디스크의 Swap

↓ 필요해짐

RAM으로 Swap In

최근 리눅스에서는 zswap도 사용할 수 있다.

[zswap]

zswap은 Swap으로 나가는 페이지를 먼저 압축해 RAM에 보관하는 캐시다.

원래 페이지 4KiB
↓ 압축
압축 페이지 1KiB
↓
RAM의 zswap 저장소

장점은 저장장치 I/O를 줄이고 Swap In 속도를 높일 수 있다는 것이다.
단점은 압축과 해제에 CPU가 필요하고, 압축된 페이지도 RAM을 사용한다는 것이다.

zswap 공간이 부족하거나 페이지가 축출되면 실제 Swap 장치로 기록될 수 있다.
zram과는 구분해야 한다.

zswap
→ 디스크로 보내질 스왑 페이지를 RAM 안에서 미리 압축해 저장하는 압축 캐시

zram
→ 물리 RAM 안에 데이터를 실시간으로 압축해 가상 스왑 공간이나 블록 장치를 만든다.

[zram]

주체:

- 리눅스 커널의 zram 드라이버
- 리눅스 커널이 페이지를 압축·해제하고 zram 장치를 관리
- 관리자 또는 시스템 설정이 zram 장치를 Swap으로 활성화

직접 대상:

- RAM에서 회수하려는 익명 메모리 페이지
- 압축하여 zram에 저장할 메모리 데이터
- RAM에 만들어진 가상 블록 장치

저장 위치:

- 압축된 데이터는 SSD/HDD가 아니라 RAM 내부의 zram 영역에 저장
- zram 장치의 관리 정보도 RAM에 저장
- zram을 Swap으로 사용하면 PTE에는 해당 zram Swap 위치가 기록됨

CPU와의 관계:

- CPU가 페이지의 압축과 압축 해제 연산을 수행
- CPU 연산 비용을 사용하는 대신 디스크 I/O를 피함
- zram 페이지에 다시 접근하면 CPU가 Page Fault를 발생
- 커널이 zram 데이터를 압축 해제하여 일반 RAM 페이지로 복구

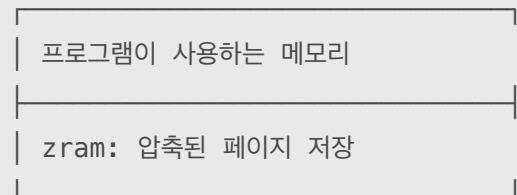
목적:

- 메모리 페이지를 압축하여 RAM을 더 효율적으로 사용
- 느린 SSD/HDD Swap 접근을 줄이거나 피함
- 디스크가 없는 장치에서도 Swap 공간을 제공
- 저용량 RAM 환경에서 OOM 발생을 늦춤

[1] zram: RAM 안에 만든 압축 Swap 장치

zram은 RAM 일부를 사용하여 압축 기능이 있는 가상 블록 장치를 만든다.

일반 RAM



이 가상 장치를 Swap으로 등록한다.

원래 페이지 4KiB

↓ 압축

압축 결과 1KiB

↓

RAM 속 zram에 저장

4KiB 페이지를 그대로 RAM에 둘 때보다 압축해서 1KiB만 사용하므로, 압축, 해제에 CPU를 조금 쓰는 대신 RAM을 절약한다.

중요한 점은 zram은 RAM을 사용하는 Swap 장치이다. SSD/HDD의 Swap 공간이 없어도 사용할 수 있다.

RAM의 일반 영역

↓ Swap-out

zram 가상 블록 장치

↓ 압축하여 RAM에 보관

끝

zram에 들어간 페이지는 기본적으로 디스크 Swap으로 자동 이동하는 구조가 아니다. 별도의 디스크 Swap도 함께 활성화했다면 커널은 zram과 디스크 Swap을 서로 다른 Swap 장치로 보고 우선순위에 따라 사용한다.

원본 페이지를 그대로 두는 것이 아닌 압축된 형태로 바꾸기 때문에 RAM을 사용해도 RAM이 늘어난다. 물리 RAM 용량 자체가 증가하는 것은 아니지만, 압축을 통해 더 많은 데이터를 RAM에 보관할 수 있다.

[2] zswap: 디스크 Swap 앞에 둔 압축 캐시

zswap은 독립적인 Swap 장치가 아니다. 기존 SSD/HDD Swap 앞에 붙는 **RAM 기반 압축 캐시**이다.

RAM 부족
↓
페이지를 Swap-out
↓
zswap에서 압축하여 RAM에 우선 보관
↓
공간 부족 또는 보관 부적합
↓
SSD/HDD의 실제 Swap에 기록

예를 들어, 4KiB 페이지를 Swap-out하려고 할 때

4KiB 페이지
↓ 압축
1KiB가 됨
↓
zswap 압축 풀에 보관

나중에 해당 페이지가 다시 필요하면 디스크까지 가지 않고 zswap에서 압축을 해제하여 빠르게 가져올 수 있다.

그러나 zswap의 압축 풀이 가득 차면 오래된 압축 페이지를 실제 디스크 Swap으로 내보낸다.

일반 RAM
↓
zswap 압축 캐시
↓ 공간 부족
SSD/HDD Swap

따라서 zswap에는 최종 저장 장소인 백엔드 Swap 장치가 필요하다.

정리하자면

디스크 Swap: 페이지를 SSD/HDD로 내보냄
zram Swap: 페이지를 압축해서 RAM 안에 보관
zswap: 디스크로 보내기 전에 RAM에 압축해 보관하는 캐시

[21] [메모리 Compaction(압축)]

주체:

- 리눅스 커널이 압축할 메모리 페이지를 선택하고 관리
- CPU가 실제 압축·압축 해제 연산을 수행
- zram, zswap 등의 커널 기능이 메모리 압축을 활용

직접 대상:

- RAM에 저장된 메모리 페이지의 데이터
- 주로 Page Reclaim 대상으로 선택된 익명 페이지

저장 위치:

- 압축 전 데이터는 일반 RAM 물리 페이지에 저장
- 압축된 데이터도 RAM의 zram 또는 zswap 영역에 저장
- 압축된 페이지의 위치와 크기 등의 관리 정보도 RAM에 저장

CPU와의 관계:

- CPU가 압축 알고리즘을 실행하여 페이지 데이터를 압축
- 페이지가 다시 필요하면 CPU가 압축을 해제
- CPU 사용량을 늘리는 대신 RAM 사용량과 디스크 I/O를 줄임
- 압축된 페이지에 접근하면 Page Fault가 발생하고 커널이 복구

목적:

- 같은 데이터를 더 적은 RAM 공간에 보관
- 디스크 Swap으로 내보내는 작업을 줄임
- RAM 부족 상황을 완화
- OOM Killer가 동작하는 시점을 늦춤
- 디스크 I/O 대신 CPU 연산을 사용해 성능 저하를 줄임

Buddy Allocator에 전체 여유 메모리는 충분하지만 연속된 큰 블록이 없을 수 있다.

사용 | 빈칸 | 사용 | 빈칸 | 사용 | 빈칸

이 상태에서는 2MiB처럼 큰 연속 메모리를 할당하기 어렵다.

리눅스는 이동 가능한 페이지를 한쪽으로 옮겨 연속된 빈 공간을 만드는 Memory Compaction을 수행할 수 있다.

[Compaction 전]

사용 | 빈칸 | 사용 | 빈칸 | 사용 | 빈칸

[Compaction 후]

사용 | 사용 | 사용 | 빈칸 | 빈칸 | 빈칸

이는 Huge Page나 연속된 물리 메모리 할당에 중요하다.

[22] [cgroup v2 메모리 관리]

프로세스 그룹별로 메모리 사용량을 계측하고 제한하며,
메모리 부족(OOM) 상황을 세밀하게 제어하는 커널 기능이다.

주체:

- 리눅스 커널의 cgroup v2 Memory Controller
- 관리자, systemd, Docker, Kubernetes 등이 제한값을 설정
- 실제 사용량 계산, Page Reclaim, 제한 적용, OOM 처리는 커널이 수행

직접 대상:

- 특정 cgroup에 소속된 프로세스들의 메모리 사용량
- 익명 메모리
- Page Cache
- 일부 커널 메모리
- Swap 사용량

저장 위치:

- 프로세스가 사용하는 실제 데이터는 RAM 또는 Swap에 저장
- 사용량, 제한값, 통계 등의 관리 정보는 RAM의 커널 메모리에 저장
- memory.max 등의 파일은 디스크 파일이 아니라 cgroupfs가 제공하는 가상 파일

CPU와의 관계:

- CPU 내부의 메모리 제한 기술은 아님
- CPU가 커널의 사용량 계산과 Page Reclaim 코드를 실행
- 프로세스의 페이지 폴트와 메모리 할당 과정에서 커널이 cgroup 제한을 확인
- CPU 사용량 제한은 memory controller가 아니라 cpu controller가 담당

목적:

- 프로세스 그룹별 RAM과 Swap 사용량을 제한
- 컨테이너나 서비스 하나가 시스템 메모리를 독점하는 것을 방지
- 중요한 서비스의 메모리를 회수로부터 보호
- 시스템 전체 OOM 대신 해당 cgroup 안에서 문제를 격리
- 프로세스 그룹별 메모리 사용량과 회수 상태를 관찰

컨테이너 환경에서는 시스템 전체의 메모리뿐 아니라 프로세스 그룹별 메모리 제한이 필요하다.
현대 리눅스는 cgroup v2의 Memory Controller를 이용한다.

대표적인 설정은 다음과 같다.

파일	의미
memory.current	현재 메모리 사용량
memory.low	가능한 한 보호할 메모리
memory.high	초과하면 회수와 제한 압력 발생
memory.max	최대 메모리 사용량
memory.swap.max	최대 Swap 사용량
memory.events	OOM 등 메모리 이벤트
memory.oom.group	cgroup 전체를 하나의 작업으로 OOM 처리

예를 들어 컨테이너의 메모리 한도를 1GiB로 제한할 수 있다.

```
echo 1G > memory.max
```

한도를 넘고 메모리 회수에도 실패하면 시스템 전체 OOM이 아니라 해당 cgroup 안에서 OOM이 발생할 수 있다.

[23] [OOM Killer]

시스템의 RAM과 스왑 메모리가 모두 소진되어 심각한 부족 상태에 빠졌을 때, 리눅스 커널이 시스템 전체의 다운(Kernel Panic)을 막기 위해 메모리를 가장 많이 쓰거나 중요도가 낮은 프로세스를 강제로 종료하여 메모리를 확보하는 비상 보호 기능이다.

주체:

- 리눅스 커널
- Page Reclaim과 메모리 확보 시도가 실패하면 커널이 실행
- cgroup 제한 때문에 발생한 경우 해당 cgroup 범위에서 실행될 수 있음

직접 대상:

- 메모리를 사용하고 있는 프로세스
- 커널이 계산한 OOM 점수가 높은 프로세스
- 선택된 프로세스가 사용하던 RAM과 Swap

저장 위치:

- 프로세스의 실제 메모리 데이터는 RAM과 Swap에 저장
- 프로세스별 메모리 사용량과 OOM 점수 관련 정보는 RAM의 커널 자료구조에 저장
- OOM 발생 정보는 커널 로그에서 확인 가능

CPU와의 관계:

- CPU 내부의 하드웨어 기능이 아님
- CPU가 리눅스 커널의 OOM 판단과 프로세스 종료 코드를 실행

- 선택된 프로세스에 일반적으로 SIGKILL을 적용
- 프로세스의 메모리 매핑이 해제되면 물리 페이지가 반환됨

목적:

- 메모리를 더 이상 확보할 수 없는 상황에서 시스템 전체가 멈추는 것을 방지
- 프로세스 하나 또는 프로세스 그룹을 종료하여 RAM을 확보
- 커널이 메모리 할당 실패 상황에서 계속 동작할 수 있도록 최후의 수단 제공
- cgroup 환경에서는 메모리 문제를 해당 서비스나 컨테이너에 격리

다음 방법으로도 메모리를 확보하지 못하면 OOM 상황이 발생한다.

- Page Cache 회수
- Dirty Page Writeback
- Swap Out
- Memory Compaction
- cgroup 내부 회수

커널은 시스템이 완전히 멈추는 것을 막기 위해 하나 이상의 프로세스를 종료해 메모리를 확보할 수 있다. 이를 OOM Killer라고 한다.

프로세스 선택에는 대략 다음 요소가 영향을 준다.

- 프로세스의 메모리 사용량
 - oom_score
 - oom_score_adj
 - cgroup 메모리 제한
 - 종료 시 확보할 수 있는 메모리
 - 보호 대상으로 설정됐는지 여부
-

[24] [malloc()부터 실제 RAM 할당까지]

사용자 프로그램에서 다음 코드가 실행됐다고 하자.

```
char *buffer = malloc(1024 * 1024);  
buffer[0] = 'A';
```

전체 흐름은 다음과 같다.

[1단계: malloc()]

malloc()은 C 라이브러리의 사용자 공간 함수다.

내부적으로 이미 확보한 Heap을 사용하거나 필요하면 brk() 또는 mmap()을 이용해 커널에 가상 주소 공간을 요청한다.

brk()

- 프로세스의 힙 메모리 영역 크기를 조절하는 시스템 콜이다.

mmap()

- 파일이나 디바이스, 혹은 메모리 공간을 프로세스의 가상 주소 공간에 연결(매핑)하는 시스템 콜이다.

```
malloc()  
↓  
기존 Heap 확인  
↓ 부족  
brk() 또는 mmap()
```

[2단계: VMA 생성 또는 확장]

커널은 프로세스의 mm_struct에 포함된 VMA를 생성하거나 확장한다.

```
Heap VMA 확장  
또는  
새 익명 mmap VMA 생성
```

이 시점에는 물리 페이지가 아직 할당되지 않을 수 있다.

[3단계: 프로세스가 실제 주소에 접근]

```
buffer[0] = 'A';
```

페이지 테이블에 물리 페이지가 연결되어 있지 않으면 Page Fault가 발생한다.

[4단계: 커널이 VMA를 검사]

커널은 접근한 주소가 유효한 VMA에 포함되는지와 쓰기 권한이 있는지를 확인한다.

```
VMA 존재?  
쓰기 가능한가?
```

[5단계: 물리 페이지 할당]

커널은 Per-CPU Pageset 또는 Buddy Allocator에서 물리 페이지를 확보한다.

[per-CPU pageset(PCP)]

멀티코어 시스템에서 각 CPU 코어별로 독립적으로 유지하는 물리 메모리 페이지 캐시 공간이다.

단일 페이지(Order 0) 메모리 요청 시 중앙 버디(Buddy) 락 경합을 방지하고 성능을 극대화한다.

[6단계: 페이지 테이블 갱신]

가상 페이지와 물리 페이지 프레임을 PTE로 연결한다.

Virtual Page → Physical Page Frame

[7단계: 명령어 재실행]

Page Fault 처리가 끝나면 중단됐던 쓰기 명령을 다시 실행한다.

buffer[0] = 'A' 성공